

Raritone — Updated Backend Architecture Documentation

Prepared By

NANDINI BHIMINENI

Project Overview

The Raritone backend architecture was redesigned and optimized to support a scalable, modular, maintainable, and production-ready system.

The architecture improvements focused on:

- Backend modularization
- Server organization
- Middleware separation
- Route management
- Error handling
- Logging & monitoring
- API scalability
- Production-ready backend flow

The backend is designed to support:

- Authentication system
- Ecommerce product management
- Virtual try-on workflows
- AI integration
- Cloud image handling
- Cart & order management
- User measurement systems
- Future scalability

Architecture Goals

The backend restructuring was performed with the following goals:

1. Scalability

Support future growth of:

- Users
- Products
- AI requests
- Image uploads
- API traffic

2. Maintainability

Improve:

- Code organization
- Debugging
- Module separation
- Team collaboration

3. Production Readiness

Prepare the backend for:

- Cloud deployment
- Load balancing
- API monitoring
- Future scaling

Backend Folder Structure

Current Structure

Raritone-Project-Backend/

|— config/

|— controllers/

|— middleware/

|— models/

|— routes/

|— uploads/

|— utils/

|— app.js

|— server.js

|— package.json

|— .env

Core Architecture Separation

app.js

Responsibilities

- Middleware configuration
- API route organization
- Compression setup
- Logging setup
- Centralized error handling
- Request parsing
- Backend application setup

Middleware Configured

CORS

Used to:

- Allow frontend-backend communication
- Support cross-origin requests

Express JSON Parser

Used for:

- Parsing request bodies
- Handling API JSON payloads

Compression Middleware

Used to:

- Improve response speed
- Reduce payload size
- Optimize performance

Morgan Logging

Used for:

- Request logging
- Backend monitoring
- Debugging APIs

Error Middleware

Used for:

- Centralized error handling
- Standardized error responses
- Debugging support

server.js

Responsibilities

- Environment configuration
- MongoDB connection
- Server startup
- Database index synchronization

Architecture Improvement

Previously

App configuration, routes, middleware, and server startup were all combined into a single file.

Now

- app.js handles application logic
- server.js handles server initialization

Benefits

- Improved scalability
- Cleaner code structure
- Better maintainability
- Easier debugging

Request Flow Architecture

Client



Routes



Middleware



Controllers



Models



MongoDB

Additional Services

- Cloudinary
- JWT Authentication
- Logging System
- Error Middleware
- Compression Middleware

Middleware Architecture

Authentication Middleware

Purpose

Protect secure APIs.

Responsibilities

- Token verification
- User authentication
- Protected route access
- Authorization handling

Error Middleware

Purpose

Handle backend errors centrally.

Handles

- Validation errors
- MongoDB errors
- JWT errors
- Invalid routes
- Internal server errors

Compression Middleware

Purpose

Improve backend performance.

Benefits

- Smaller responses
- Faster APIs
- Reduced bandwidth usage

Logging Middleware

Morgan Logging

Used for:

- API request tracking
- Debugging
- Backend monitoring
- Performance analysis

Route Organization

The backend APIs were modularized into dedicated route files.

Route Structure

Authentication Routes

/api/auth

Handles:

- Registration
- Login
- Authentication

Product Routes

/api/products

Handles:

- Product management
- Product retrieval

Profile Routes

/api/profile

Handles:

- User profile management

Wishlist Routes

/api/wishlist

Handles:

- Wishlist operations

Wardrobe Routes

/api/wardrobe

Handles:

- Wardrobe management

Try-On Routes

/api/tryOnRoutes

Handles:

- AI try-on workflows
- Virtual try-on requests

Measurement Routes

/api/measurements

Handles:

- User measurements
- Body size management

Cart Routes

/api/cart

Handles:

- Cart management
- Quantity updates
- Product removal

Order Routes

/api/orders

Handles:

- Order management
- Shipping details
- Order tracking

Image Routes

/api/images

Handles:

- Image uploads
- Image APIs
- Cloud image handling

Backend Modules Integrated

The backend now supports the following modules:

Authentication Module

Features

- JWT authentication
- Protected routes
- Secure login system

Product Module

Features

- Product listing
- Product APIs
- Scalable product structure

Measurement Module

Features

- User body measurements
- Scalable schema structure
- Measurement retrieval

Cart Module

Features

- Add to cart
- Update quantities
- Remove products
- Clear cart

Order Module

Features

- Order creation
- Shipping details
- Payment method support
- Order status updates

Wishlist Module

Features

- User wishlist management

Try-On Module

Features

- AI-ready try-on architecture

- Image request handling

Database Architecture Improvements

Improvements Implemented

- ObjectId references
- Timestamps
- Indexing
- Modular collections
- Relationship optimization
- Scalable schema structure

Collections Used

- Users
- Products
- Orders
- Measurements
- Wishlist
- Cart
- TryOn
- Wardrobe

Security Improvements

Implemented

- Protected APIs
- Middleware organization
- Centralized error handling
- Secure environment variable handling

Research Conducted

- Helmet
- Rate limiting
- Google OAuth

- Scalable API protection

Scalability Improvements

Implemented

- Modular architecture
- Route separation
- Centralized middleware
- Scalable backend structure
- Logging setup
- Compression support

Future Scalability Plans

Research Areas

- Redis
- Docker
- AWS deployment
- Load balancing
- Monitoring systems
- CDN optimization

Cloud & File Handling Support

Current/Future Support

- Cloudinary integration
- Secure uploads
- Image workflow optimization
- AI image handling
- Future AWS S3 support

Production Readiness Improvements

The backend was improved to support:

- Modular development
- Team collaboration

- Scalable deployment
- Production-level debugging
- Maintainable architecture

Implemented Improvements

- app.js separation
- Clean server.js
- Middleware organization
- Route modularization
- Centralized error handling
- Logging & compression
- Scalable database structure

Architecture Diagram

Frontend Client



Backend API Routes



Middleware Layer



Controllers



MongoDB + Cloudinary



Future AI Services

Future Improvements

Planned Backend Enhancements

- Redis caching
- Docker containerization
- Kubernetes orchestration

- AI microservices
- API gateway architecture
- Cloud deployment optimization
- CDN support
- Advanced monitoring

Final Summary

The Raritone backend architecture was successfully optimized into a scalable, modular, and production-ready backend structure.

Major Architecture Improvements Completed

- Backend modularization
- app.js / server.js separation
- Middleware organization
- Centralized error handling
- Logging system integration
- Compression support
- Scalable API structure
- Database optimization support
- Production-ready backend flow

Backend Ready For

- Frontend integration
- AI virtual try-on integration
- Scalable ecommerce workflows
- Future cloud deployment
- Future infrastructure scaling

Conclusion

The updated backend architecture establishes a strong technical foundation for the Raritone platform. The system is now structured for scalability, maintainability, security, and future AI-driven ecommerce expansion.

The modular backend design ensures easier collaboration between development teams while also preparing the platform for production deployment, cloud scalability, and future enterprise-level enhancements.